

# Phase 1 - Architecture Flow

## Input - Prof's course idea (same as in Gems)



# Steps Defined (1–6)

## STEP 1

### Understand Request

- Convert form to a structured record without inventing details.
- Fill missing values using official IIIT-D regulations (13 weeks, 3L+1T for 4 credits).
- Explicitly list all assumed values for professor confirmation.

## STEP 2

### Curriculum Placement

- Identify overlapping IIIT-D courses, prerequisites, and duplicate risks.
- Benchmark against top global universities.
- Filter strictly for verified single-course pages (excluding department pages).

## STEP 3

### Rule Conversion

- Translate professor preferences directly into testable rules.
- Flag regulatory conflicts explicitly instead of making silent decisions.

## STEP 4

### Outcomes & Alignment

- Course outcomes defined clearly with measurable actions, tools, and level.
- Linked directly to official IIIT-D programme outcomes with explicit rationale.
- Broken into 1–3 week steps tested by single tasks and tied to lectures.

## STEP 5

### Structure & Timetable

- Topics placed across 13 weeks.
- Explicitly identifies and flags overflow topics that must be cut to fit capacity.

## STEP 6

### Assessments & Rationale

- Quizzes, assignments, and exams state exact tested outcomes and mark shares.
- Strict constraint: no material is tested prior to being explicitly taught.
- Provides end-to-end proof that all taught material is assessed.

# Steps Defined (7–8) & Quality Control

## STEP 7

### Reading Material

- Verified library databases (OpenLibrary, Crossref) for books and papers.
- Title, author, and year strictly checked.
- AI cannot invent citations or references.

## STEP 8

### Final Proposal

- Auto-produced in official IIIT-D course-sheet format.
- Includes assumptions, mapping, timetable, & comparable courses.
- Includes open issues report for full transparency.

## QUALITY CONTROL

### Automated Rule Checks

- Rules enforced as code after every single step.
- Vague verbs ("understand", "know") strictly banned.
- Ensures no material is tested before being explicitly taught.

## REPAIR LOOP

### Fix & Escalation

- AI receives up to 3 attempts to repair failed checks.
- A fix is retained only if it reduces total error count.
- Unfixed errors automatically escalate to the professor.

## GOVERNANCE

### State Locking

- Professor-approved parts are locked immediately.
- Locked sections cannot be changed by subsequent steps.
- Guarantees final course sheet never drifts from design.

## AI CONSTRAINTS

### Core Instructions

- Cite source of every fact; never invent codes or refs.
- Never self-claim correctness without verification.
- Powered by Gemini Flash models for fast execution.

## Example Output - [Detailed Output Link](#)

### Post Conditions (Course Outcomes)

| CO  | Students will be able to...                                                                                                                                                                  |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CO1 | Configure coding agents and agent skills, using an agent orchestration framework and Model Context Protocol, such that at least 80% of the provided test cases pass.                         |
| CO2 | Construct executable specifications from a given natural-language requirements document, using spec-driven development tools, covering every use case in the supplied requirements document. |
| CO3 | Analyze execution traces produced by coding agents during iterative loop engineering, identifying at least three distinct failure causes with supporting log evidence.                       |
| CO4 | Evaluate automated testing and reliability frameworks for agent-generated code, justifying each evaluation decision against at least one stated requirement.                                 |
| CO5 | Evaluate a complete AI-native software development workflow for a given multi-module project, demonstrated live with two scenario walkthroughs chosen by the evaluator.                      |

[Github Link](#) (contains detailed checks as well)

# Defect Validation Framework

## PROBLEM STATEMENT

### Why Validation?

AI-generated course designs can appear well-written while containing logical defects such as:

- **Assessments scheduled** before relevant topics are taught
- Assessment weights not summing to **100%**
- Vague / non-checkable learning outcomes

## VALIDATION ENGINE

### How It Works

- **Deterministic Python validators** inspect every generated stage
- **No LLM** is used to judge another LLM's output

Violations represented as structured **error/warning objects**:

- Rule code & severity | Affected nodes
  - Human-readable message | Evidence + repair hint

## 9-Stage Validation Pipeline

1. Course Context → 2. Prerequisites → 3. Constraints → 4. CO/PO/LOs → 5. Schedule → 6. Assessment → 7. Resources → 8. Proposal

## Key Validation Checks

### Outcomes

Bloom levels, verbs, CO-LO relationships, duplicate detection

### Schedule

Teaching hours, topic ordering sequence, LO coverage verification

### Assessment

100% weight summation, Bloom alignment, temporal prerequisite ordering

### Resources

API-based citation and bibliography verification

# Automated Repair & Strict Acceptance

From Detection → Repair → Verification

## STEP 1

### 1. Detect

Validators identify concrete violations such as **V19**:

Assessment in Week 7 tests an LO whose topics continue until Week 8.

## STEP 2

### 2. Repair (R1 Agent)

The repair agent receives:

- Exact violations
- Affected nodes
- **Fix hints**

## STEP 3

### 3. Re-validate

A repair is accepted **only if**:

- **Errors after < Errors before**
- **No new violation types**

Otherwise rejected; previous version retained.

## REPAIR BEHAVIOR

### Example Repair Sequence

10 errors → 5 → 1 → 0

**ACCEPT → ACCEPT → ACCEPT**

1 error → 2 errors

**REJECT → retry**

## TERMINATION RULES

### Final-State Logic

- Fixed within 3 rounds → **repaired**
- Survives all 3 rounds → **escalated & saved**
- Warning → **flagged & saved**
- Worse patch → **old version retained**

**Core Idea:** Validation is not just detection; it creates a **closed-loop generate → verify → repair → verify system**.

# Mutation Benchmark & Results

Defect Benchmark: Does the Validator Actually Catch Errors?

## Methodology

### Mutation Testing

Clean CS-601 YAML



Inject controlled defect



Run ProfsAgent pipeline



Check surviving violations

$DIR = \text{Defects Detected} / \text{Injected}$

## INJECTED DEFECT RESULT

130% assessment weights ✓ Detected

Banned "understand" verb 🔧 Repaired

Bloom L1 vs L6 mismatch ✓ Detected

Orphan learning outcomes ✓ Detected (6 violations)

Invalid LO assessment IDs ✓ Detected

Assessment before teaching ✓ Detected

## ANALYSIS

### Key Findings

- **Repair loop is critical:** some defects are fixed before reaching final state.
- **Cascading failures are revealing:** one bad input can trigger multiple independent validator violations.
- **Deterministic rules constrain LLM failures:** even deliberately injected contradictions were caught or repaired.